

Introduction to Evolutionary Computation

- Evolutionary Computation is the field of study devoted to the design, development, and analysis of problem solvers based on natural selection (simulated evolution).
- Evolution has proven to be a powerful search process.
- Evolutionary Computation has been successfully applied to a wide range of problems including:
 - Aircraft Design,
 - Routing in Communications Networks,
 - Tracking Windshear,
 - Game Playing (Checkers [Fogel])

Introduction to Evolutionary Computation

- Evolutionary computation uses the computational model of evolutionary processes as key elements in the design and implementation of computer-based systems and problem solving applications.
- There are a variety of evolutionary computational models that have been proposed and studied which we will refer to as evolutionary algorithms.
- They share a common conceptual base of simulating the evolution of individual structures via processes of selection and reproduction.
- They depend on the performance (fitness) of the individual structures.

Introduction to Evolutionary Computation - Example

```
Procedure EC{  
  t = 0;  
  Initialize Pop(t);  
  Evaluate Pop(t);  
  While (Not Done)  
  {  
    Parents(t) = Select_Parents(Pop(t));  
    Offspring(t) = Procreate(Parents(t));  
    Evaluate(Offspring(t));  
    Pop(t+1)= Replace(Pop(t),Offspring(t));  
    t = t + 1;  
  }
```

Introduction to Evolutionary Computation

- In an Evolutionary Computation, a population of candidate solutions (CSs) is randomly generated.
- Each of the CSs is evaluated and assigned a fitness based on a user specified evaluation function. The evaluation function is used to determine the 'goodness' of a CS.
- A number of individuals are then selected to be parents based on their fitness. The *Select_Parents* method must be one that balances the urge for selecting the best performing CSs with the need for population diversity.

Evolutionary Computation - Applications

- Robotics
- Air Traffic Control
- Design
- Scheduling
- Machine Learning
- Pattern Recognition
- Strike Force Allocation

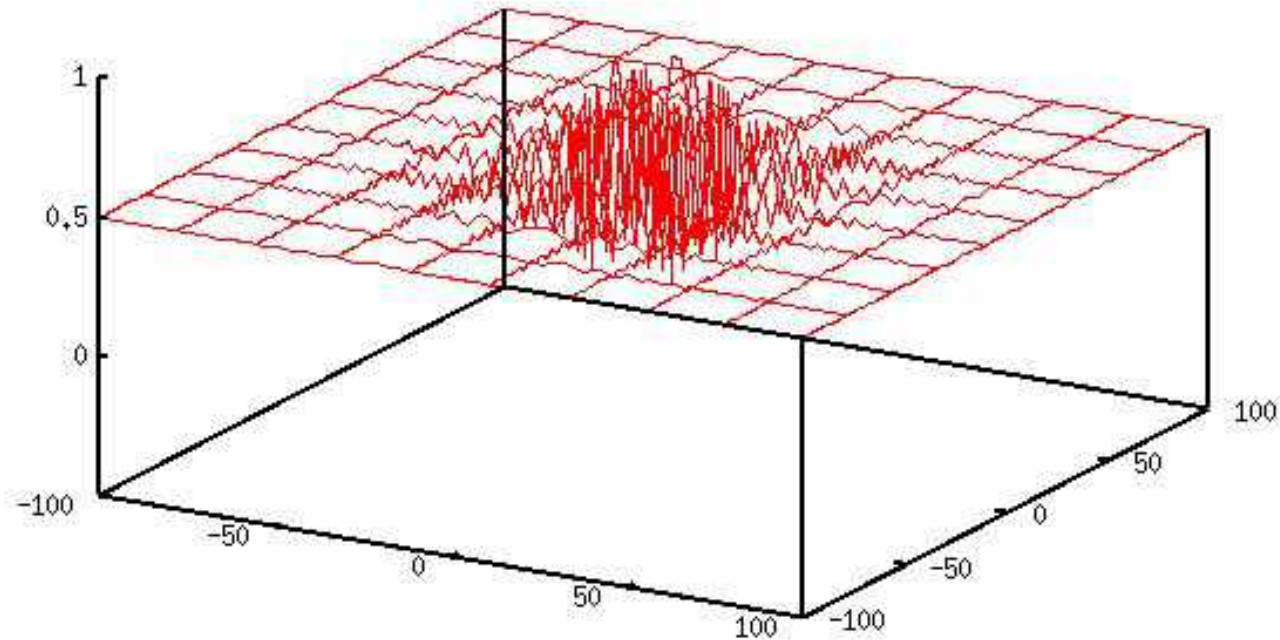
Introduction to Evolutionary Computation: A Simple Example

- Let's walk through a simple example!
- Let's say you were asked to solve the following problem:
 - Maximize:
 - $f(x,y) = 0.5 + (\sin(\sqrt{x^2+y^2}))^2 - 0.5 / (1.0 + 0.001(x^2+y^2))^2$
 - Where x and y are taken from [-100.0,100.0]
 - You must find a solution that is greater than 0.99754, and
 - you can only evaluate a total of 4000 candidate solutions (CSs)
- This seems like a difficult problem. It would be nice if we could see what it looks like! This may help us determine a good algorithm for solving it.

Introduction to Evolutionary Computation: A Simple Example

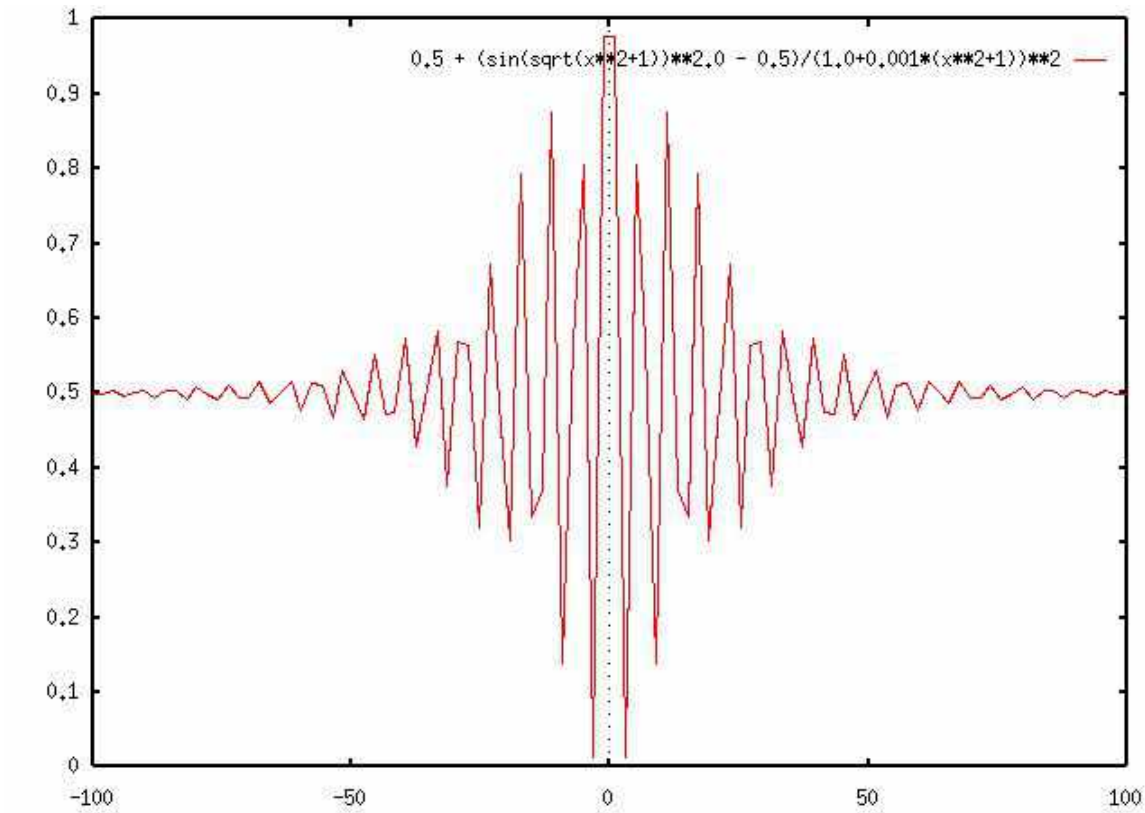
- A 3D view of $f(x,y)$:

$$0.5 + (\sin(\sqrt{x^2+y^2}))^2 \cdot 0.5 / (1.0 + 0.001 \cdot (x^2+y^2))^2$$



Introduction to Evolutionary Computation: A Simple Example

- If we just look at only one dimension $f_6(x, 1.0)$



Introduction to Evolutionary Computation: A Simple Example

```
Procedure simpleEC{
    t = 0;
    Initialize Pop(t); /* of P individuals */
    Evaluate Pop(t);
    while (t <= 4000-P){
        Select_Parent(<xmom, ymom>); /* Randomly */
        Select_Parent(<xdad, ydad>); /* Randomly */
        Create_Offspring(<xkid, ykid>):
            xkid = rnd(xmom, xdad) + Nx(0,σ);
            ykid = rnd(ymom, ydad) + Ny(0,σ);
            fitkid = Evaluate(<xkid, ykid>);
            Pop(t+1) = Replace(worst, kid); {Pop(t)-{worst}} ∪ {kid}
            t = t + 1;
    }
}
```

Evolutionary algorithms (EA)

- More precisely, evolutionary algorithms maintain a population of structures that evolve according to rules of selection and other operators, such as recombination and mutation.
- Each individual in the population receives a measure of its fitness in the environment.
- Selection focuses attention on high fitness individuals, thus exploiting the available fitness information.

Evolutionary algorithms (EA)

- Recombination and mutation perturb those individuals, providing general heuristics for exploration.
- Although simplistic from a biologist's viewpoint, these algorithms are sufficiently complex to provide robust and powerful adaptive search mechanisms.
- A population of individual structures is initialized and then evolved from generation to generation by repeated applications of evaluation, selection, recombination, and mutation.
- The population size N is generally constant in an evolutionary algorithm

Evolutionary algorithms (EA)

- An evolutionary algorithm typically initializes its population randomly, although domain specific knowledge can also be used to bias the search.
- Evaluation measures the fitness of each individual according to its worth in some environment.
- Evaluation may be as simple as computing a fitness function or as complex as running an elaborate simulation
- Selection is often performed in two steps, parent selection and survival.

Evolutionary algorithms (EA)

- Parent selection decides who becomes parents and how many children the parents have.
- Children are created via recombination, which exchanges information between parents, and mutation, which further perturbs the children.
- The children are then evaluated. Finally, the survival step decides who survives in the population.

Evolutionary programming (EP)

- Developed by Fogel (1966), and traditionally has used representations that are tailored to the problem domain.
- For example, in real-valued optimization problems, the individuals within the population are real-valued vectors.
- Other representations such as ordered lists, and graphical representations could be applied depending on the problem itself.

Evolutionary programming (EP)

- After initialization, all N individuals are selected to be parents, and then are mutated, producing N children.
- These children are evaluated and N survivors are chosen from the $2N$ individuals, using a probabilistic function based on fitness.
- In other words, individuals with a greater fitness have a higher chance of survival.
- The form of mutation is based on the representation used.

Evolutionary programming (EP)

- For example, when using a real-valued vector, each variable within an individual may have an adaptive mutation rate that is normally distributed with a zero expectation.
- Recombination is not generally performed since the forms of mutation used are quite flexible and can produce perturbations similar to recombination, if desired.

Learning Classifier Systems

An LCS is a rule-based system that incorporates evolutionary computing and machine learning to solve a given task, by interacting with the environment. A basic

LCS consists of

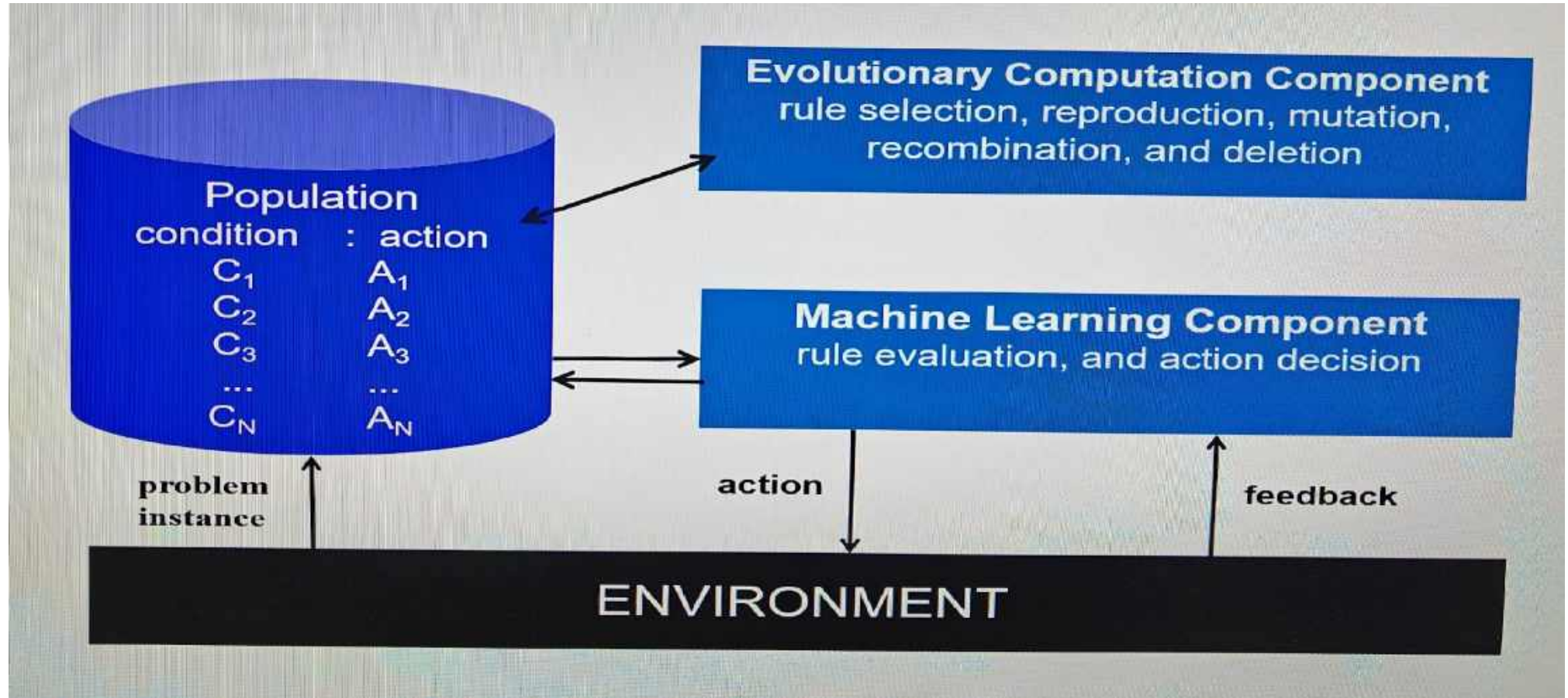
- (1) a set of rules, that is a population of classifiers,
- (2) a rule evaluation mechanism, which usually is realized by adapted RL techniques, and
- (3) a rule evolution mechanism, which is usually implemented by a GA.

The classifier population codes the current knowledge of the LCS. The evaluation mechanism estimates (and propagates) rule utility. Based on the estimated utilities, the evolutionary mechanism generates offspring classifiers and deletes less useful classifiers.

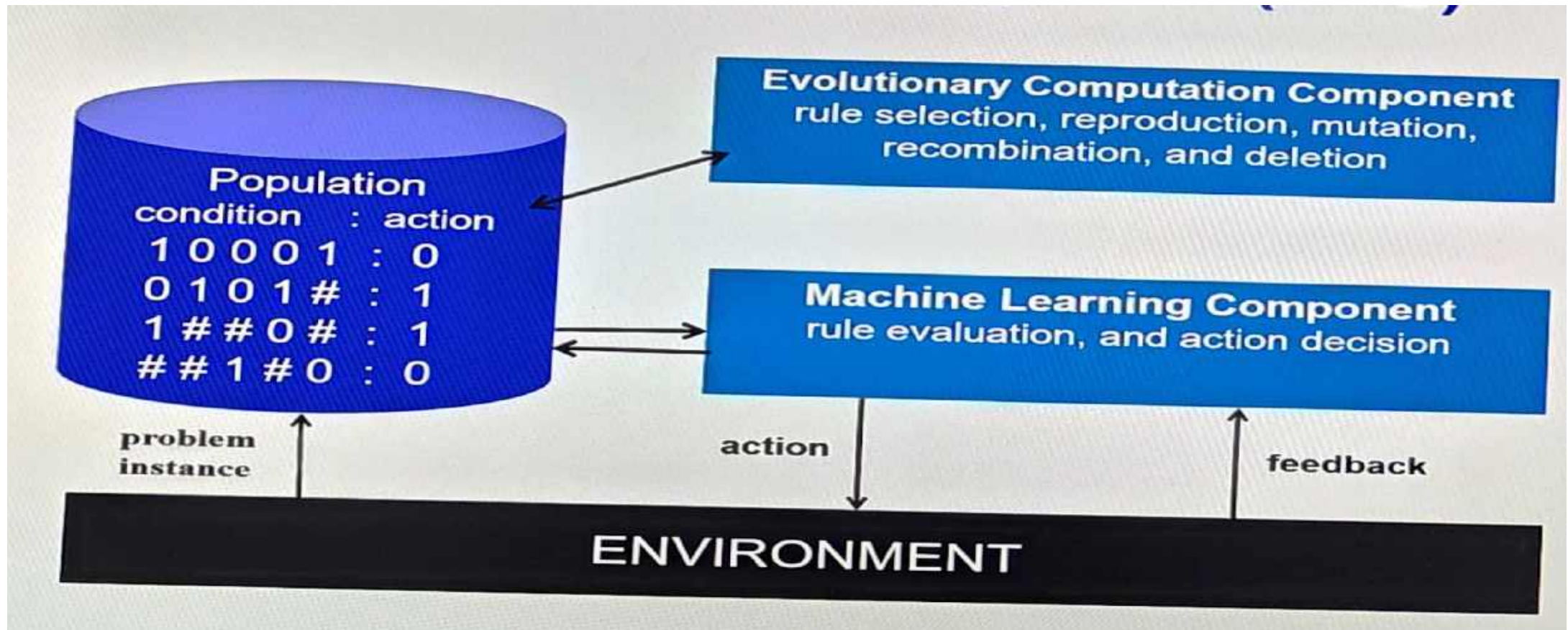
Learning Classifier Systems

- A learning classifier system (LCS) is an adaptive system that learns to perform the best action given its input. By "best" is generally meant the action that will receive the most reward or reinforcement from the system's environment. Payoff received for a given action is used by the LCS to alter the likelihood of taking that action, in those circumstances, in the future. By "input" is meant the environment as sensed by the system, usually a vector of numerical values. The set of available actions depends on the decision context, for instance a financial one, the actions might be "buy", "sell", etc.
- In general, an LCS is a simple model of an intelligent agent interacting with an environment.

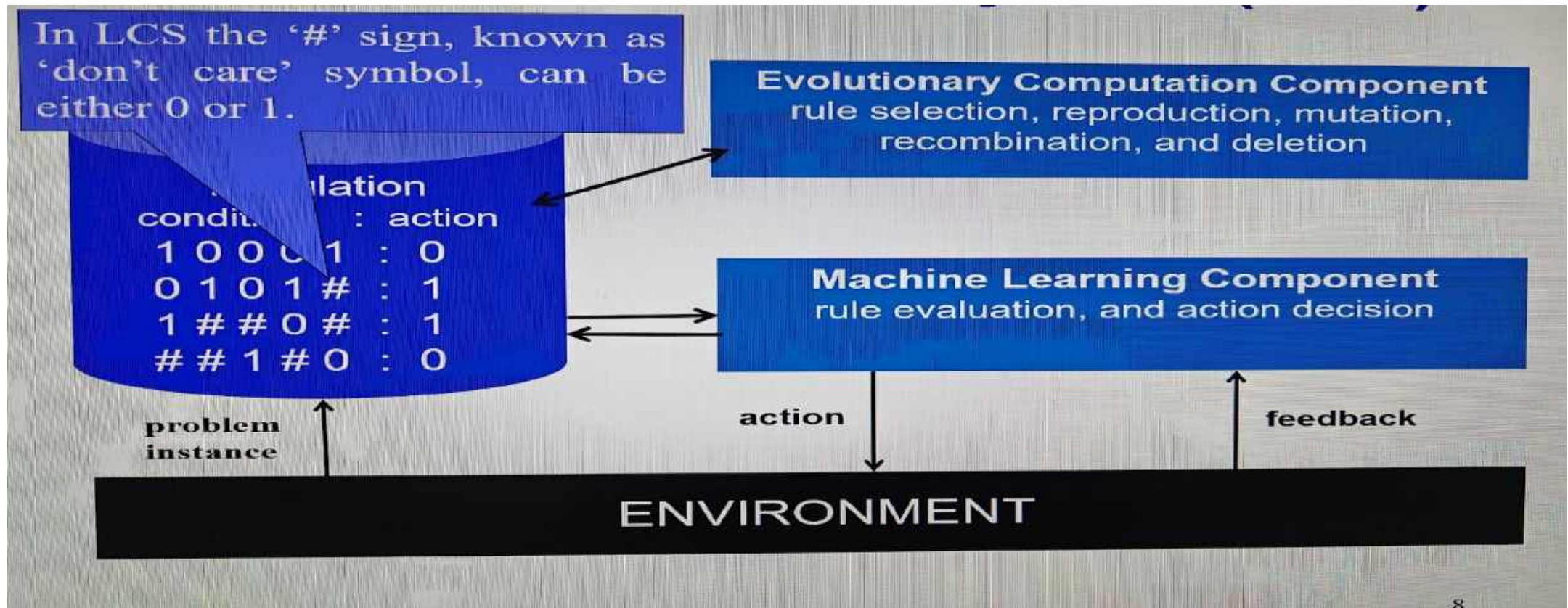
Working of LCS



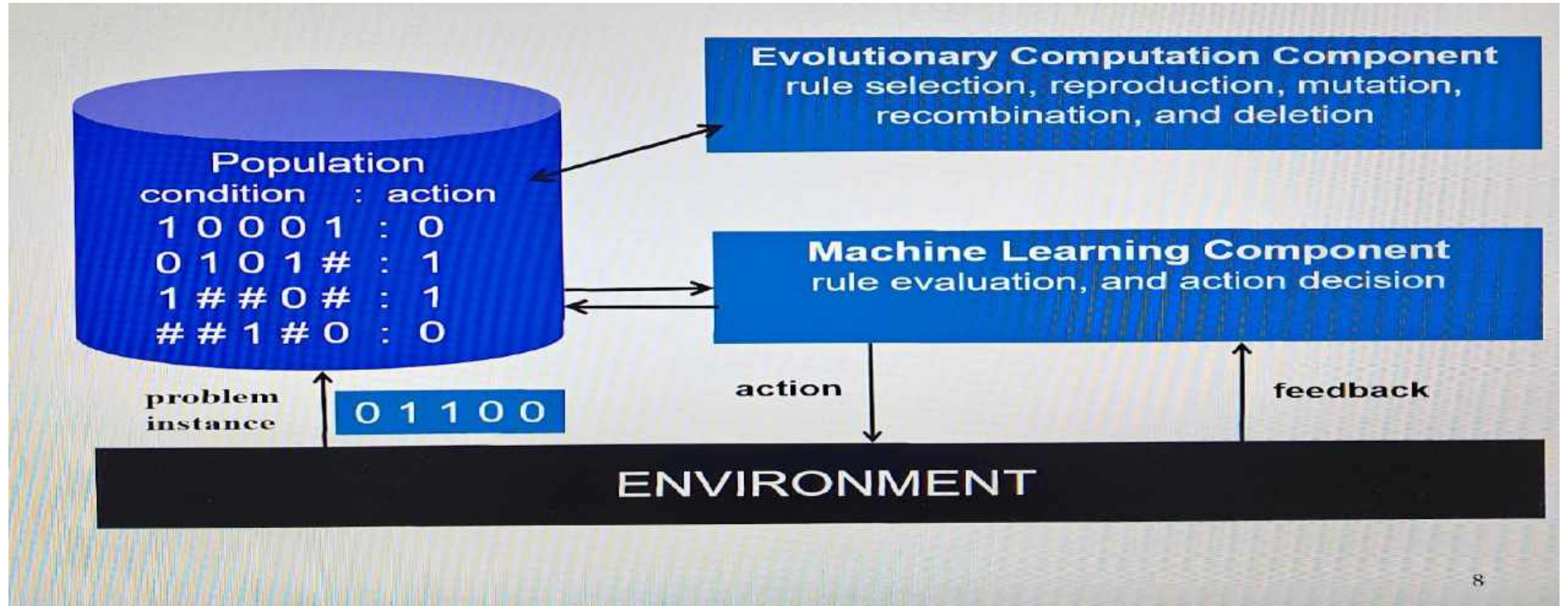
Working of LCS



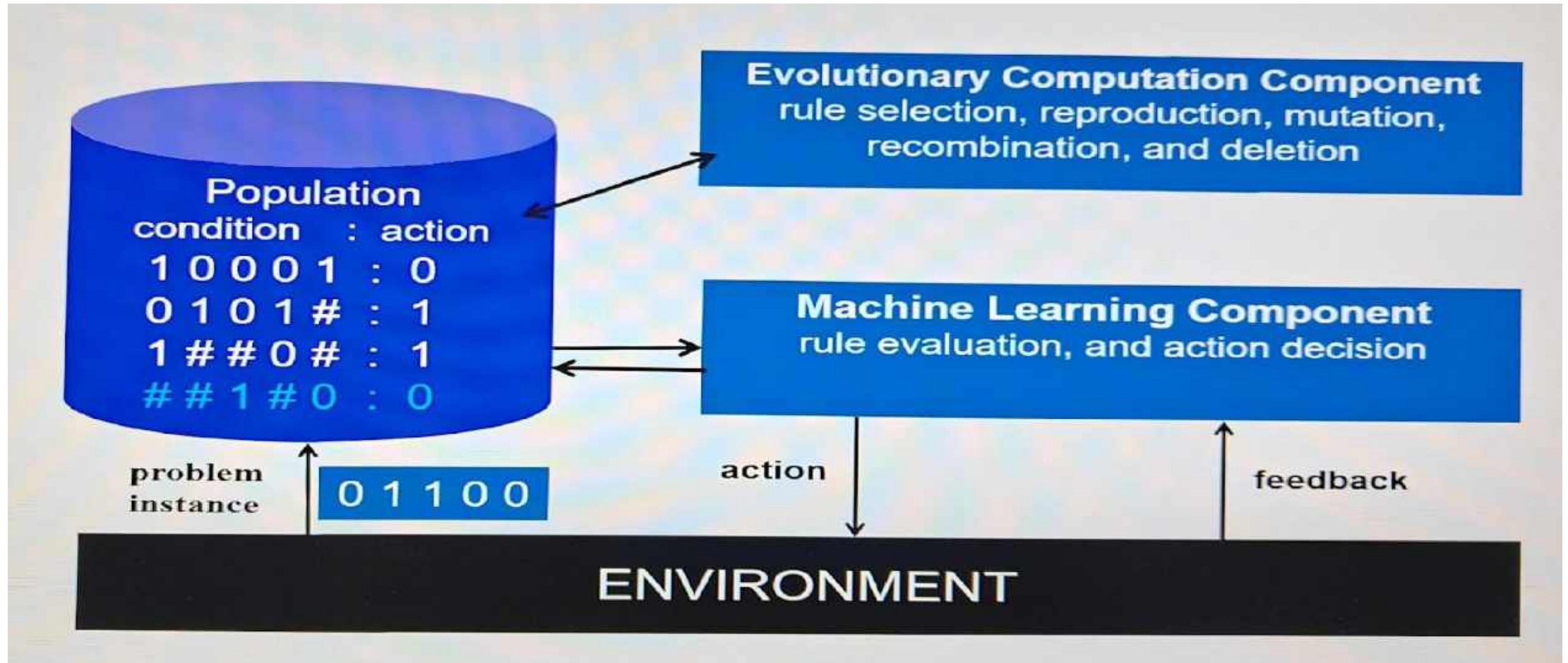
Working of LCS



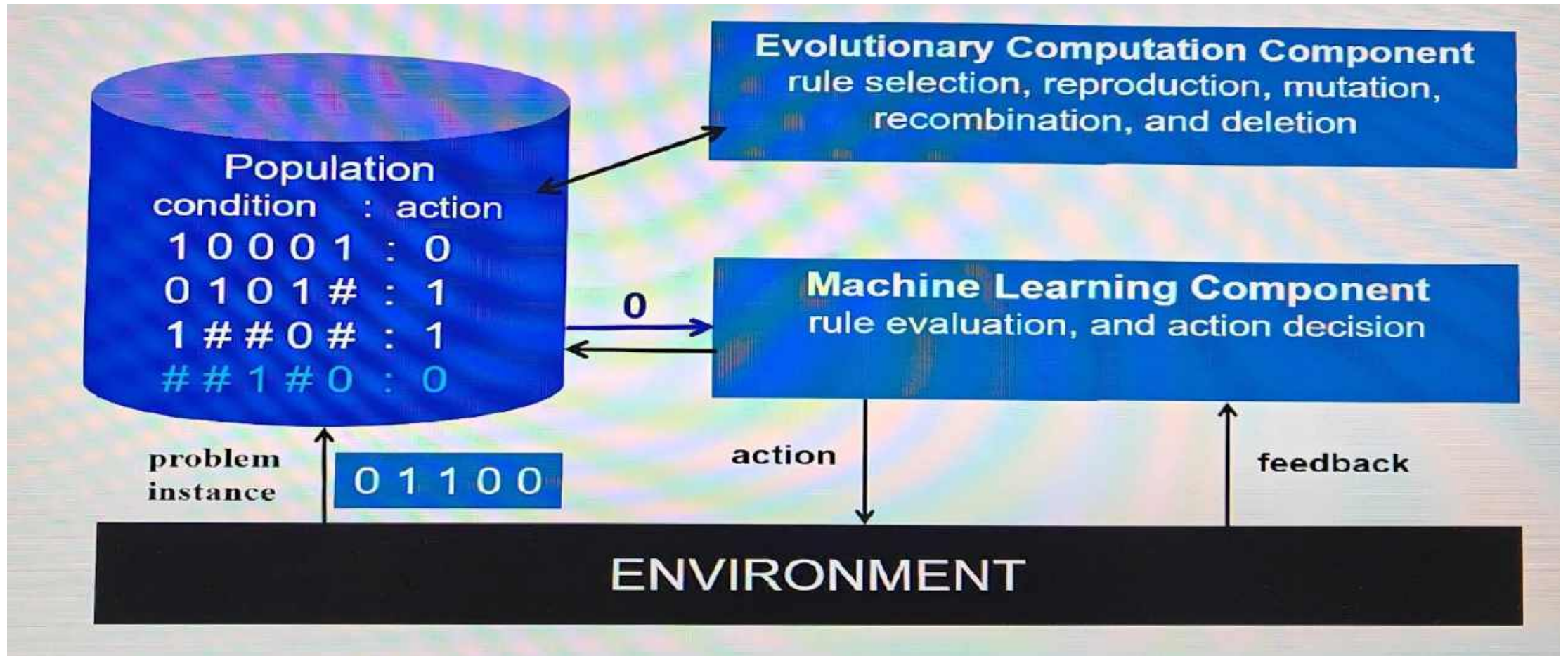
Working of LCS



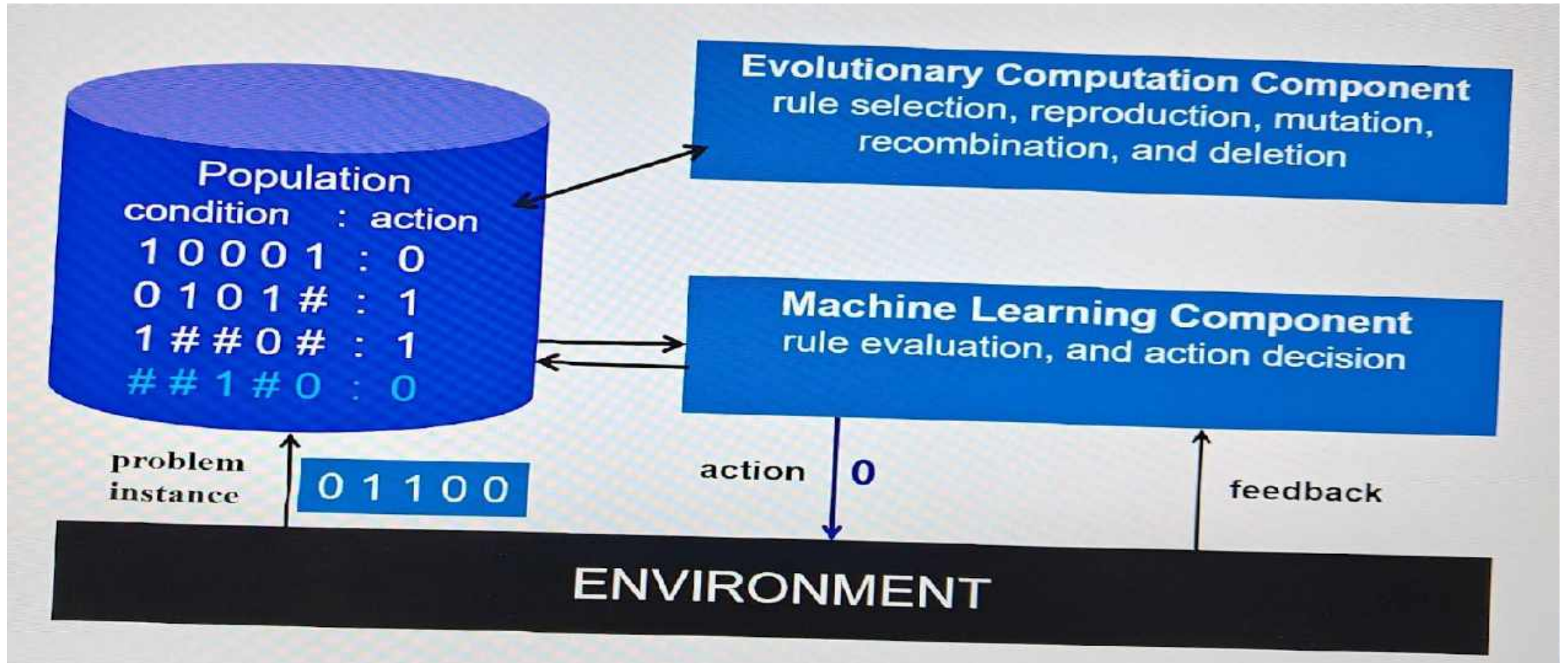
Working of LCS



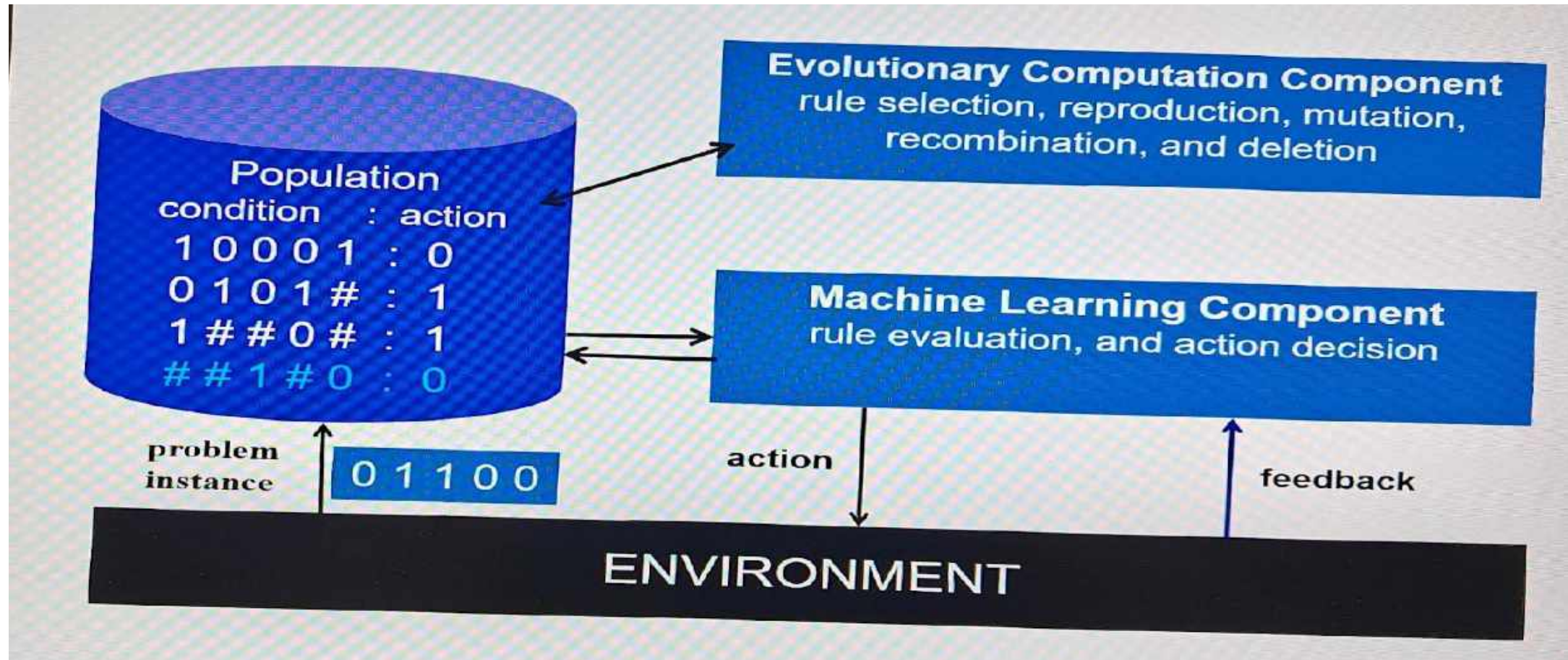
Working of LCS



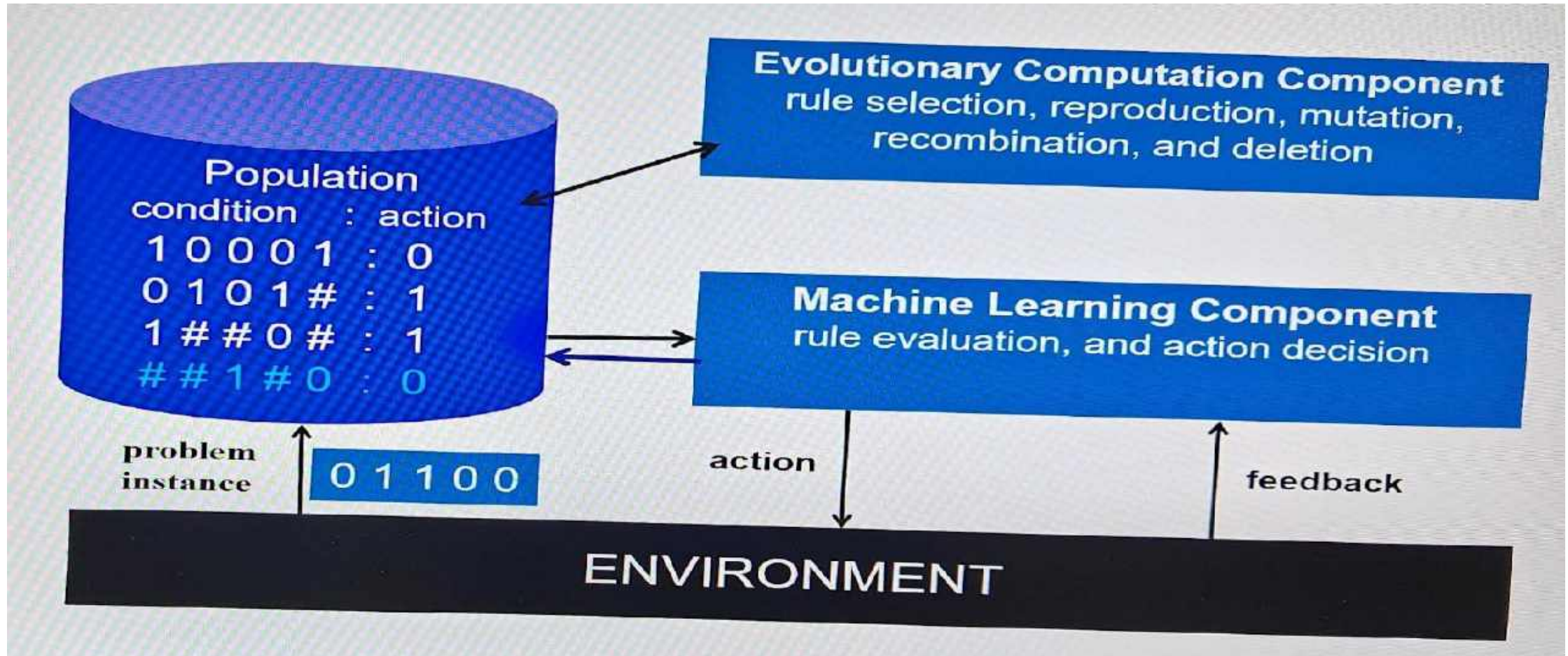
Working of LCS



Working of LCS



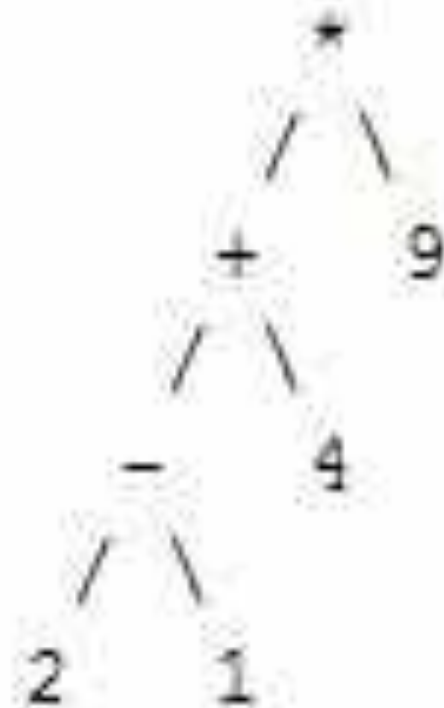
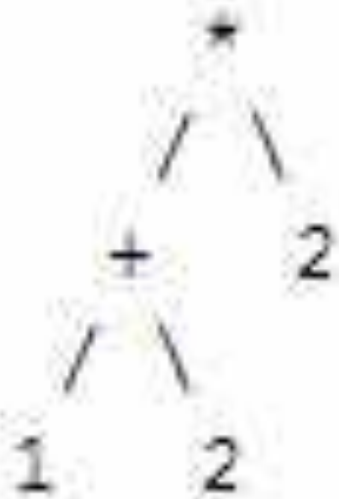
Working of LCS



Genetic Programming (GP)

- In programming languages such as LISP, the mathematical notation is not written in standard notation, but in prefix notation. Some examples of this:
- $+ \ 2 \ 1$: $2 + 1$
- $* \ + \ 2 \ 1 \ 2$: $2 * (2 + 1)$
- $* \ + \ - \ 2 \ 1 \ 4 \ 9$: $9 * ((2 - 1) + 4)$
- Notice the difference between the left-hand side to the right? Apart from the order being different, no parenthesis! The prefix method makes it a lot easier for programmers and compilers alike, because order precedence is not an issue.
- You can build expression trees out of these strings that then can be easily evaluated, for example, here are the trees for the above three expressions.

GP– Tree Structure Representation



GP

- A genetic algorithm can be used to 'evolve' an expression tree to create a very close fit to the data.
- By 'splicing' and 'grafting' the trees and evaluating the resulting expression with the data and testing it to the answers, the fitness function can return how close the expression is.

GP - Flowchart

1. Randomly create an initial population (generation 0) of individual computer programs composed of the available functions and terminals.
2. Iteratively perform the following sub-steps (called a gemmation) on the population until the termination criterion is satisfied:
 - Execute each program in the population and ascertain its fitness (explicitly or implicitly) using the problem's fitness measure.
 - Select one or two individual program(s) from the population with a probability based on fitness (with re-selection allowed) to participate in the generic operations in the next sub-steps.

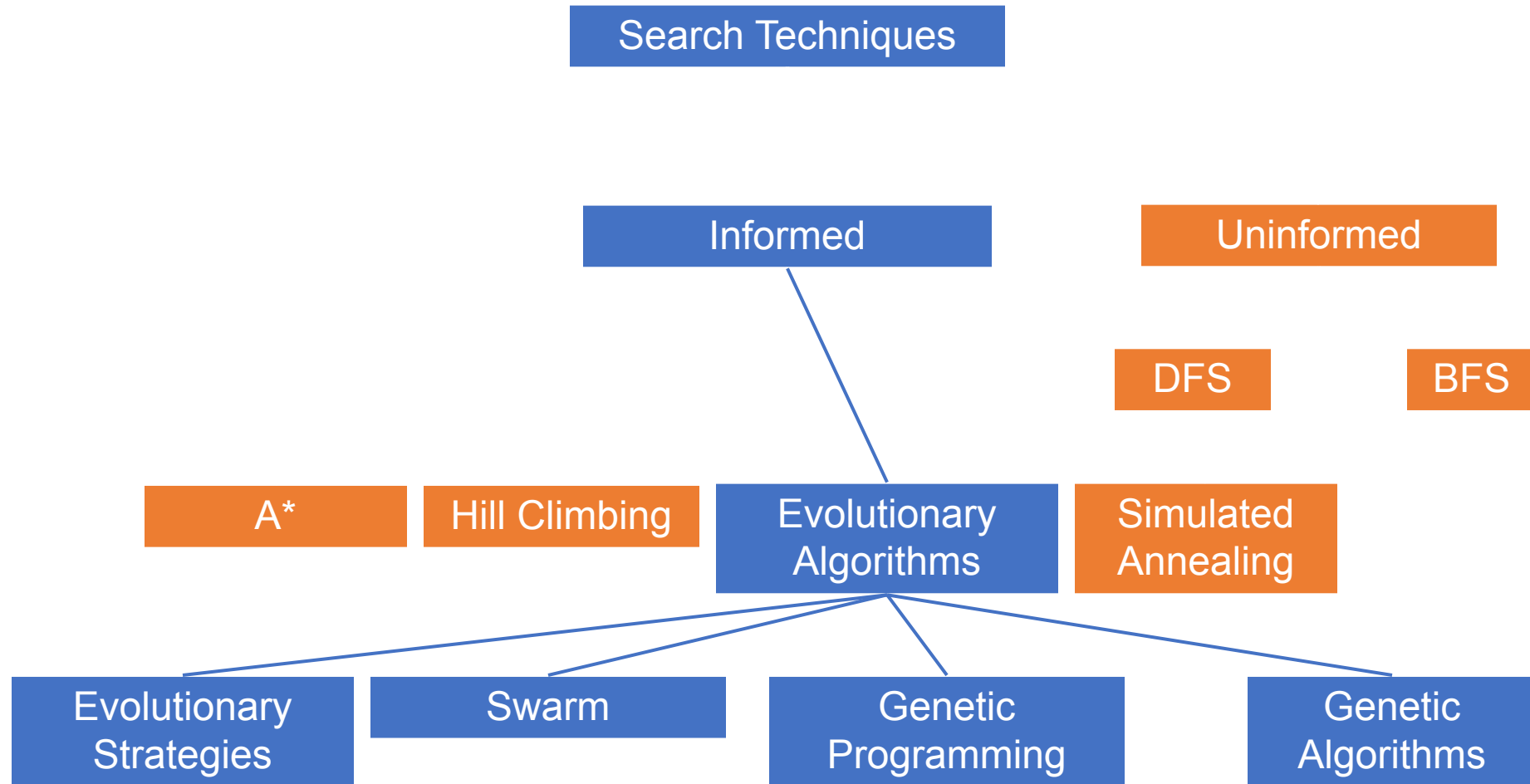
GP - Flowchart

- Create new individual program(s) for the population by applying the following generic operations with specified probabilities:
 - (a) Reproduction: Copy the selected individual program to the new population.
 - (b) Crossover: Create new offspring program(s) for the new population by recombining randomly chosen part from two selected programs.
 - (c) Mutation: Create one new offspring program for the new population by randomly mutating a randomly chosen part of one selected program.
 - (d) Architecture-Altering operation- Choose a Architecture-Altering operation from the available repertoire of such operations and create one new offspring program for the new population by applying the chosen architecture-altering operation to one selected program.
- 3. After the termination criterion is satisfied, the single best program in the population produced during the run (the best-so-far individual) is harvested and designated as the result of the run. If the run is successful, the result may be a solution (or approximate solution) to the problem.

GP

- The limitations of genetic programming lie in the **huge** search space the GAs have to search for - an infinite number of equations.
- Therefore, normally before running a GA to search for an equation, the user tells the program which operators and numerical ranges to search under.
- Uses of genetic programming can lie in stock market prediction, advanced mathematics and military applications .

Taxonomy



Genetic Algorithms

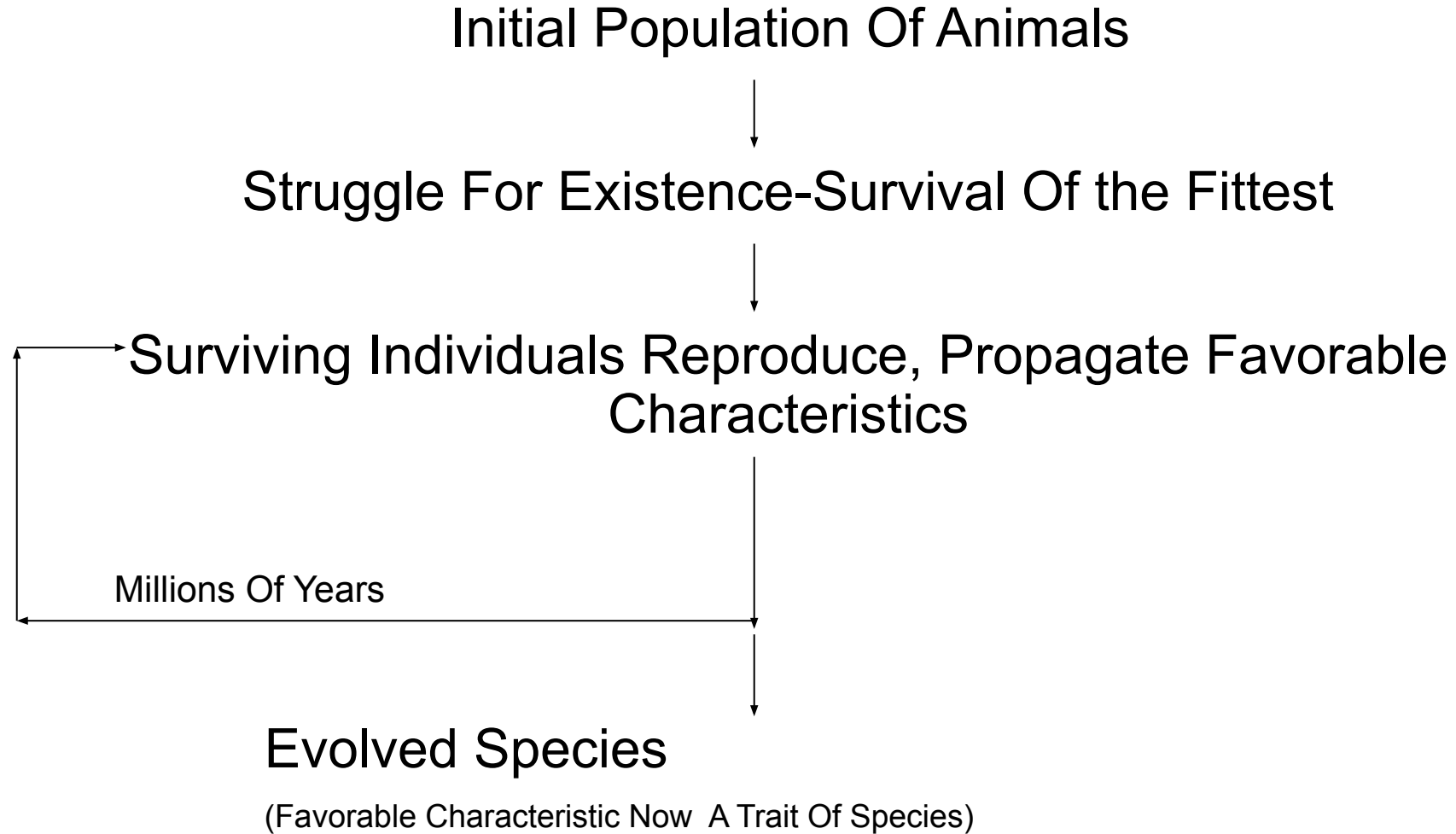
- Genetic Algorithms (GAs) are **search and optimization methods** inspired by natural evolution.
- Work with a **population of candidate solutions**, not a single solution.
- Use **evolutionary principles** like:
 - Natural selection
 - Recombination
 - Mutation
- Suitable for **complex, nonlinear, multimodal optimization problems**.

Darwin's Principle Of Natural Selection

- IF there are organisms that reproduce, and
- IF offsprings inherit traits from their progenitors, and
- IF there is variability of traits, and
- IF the environment cannot support all members of a growing population,
- THEN those members of the population with less-adaptive traits (determined by the environment) will die out, and
- THEN those members with more-adaptive traits (determined by the environment) will thrive

The result is the evolution of **species**.

Evolution Through Natural Selection



Biological Foundations

Evolutionary concepts that inspired GA:

- **Genes:** basic hereditary units
- **Chromosomes:** sequences of genes
- **Alleles:** possible gene values
- **Population:** group of chromosomes
- **Fitness:** ability to survive/reproduce

Biological processes mirrored:

- **Mitosis (copying)**
- **Meiosis (mixing)**
- **Selection (survival of the fittest)**

Nature to Computer Mapping

Nature	Computer
Population	Set of solutions.
Individual	Solution to a problem.
Fitness	Quality of a solution.
Chromosome	Encoding for a Solution.
Gene	Part of the encoding of a solution.
Reproduction	Crossover

GA vs Traditional Methods

Traditional Optimizers

- Require gradient or derivative information
- Sensitive to initial point
- Often get stuck in local minima

Genetic Algorithms

- Work on **coded variables**, not the variables themselves
- Search **in parallel** across many points
- **Global** exploration capability
- No derivative requirement
- Effective for discrete, noisy, multimodal spaces

GA - Definition

- Genetic algorithms are implemented as a computer simulation in which a population of abstract representations (called chromosomes or the genotype or the genome) of candidate solutions (called individuals, creatures, or phenotypes) to an optimization problem evolves toward better solutions.
- Traditionally, solutions are represented in binary as strings of 0s and 1s, but other encodings are also possible.

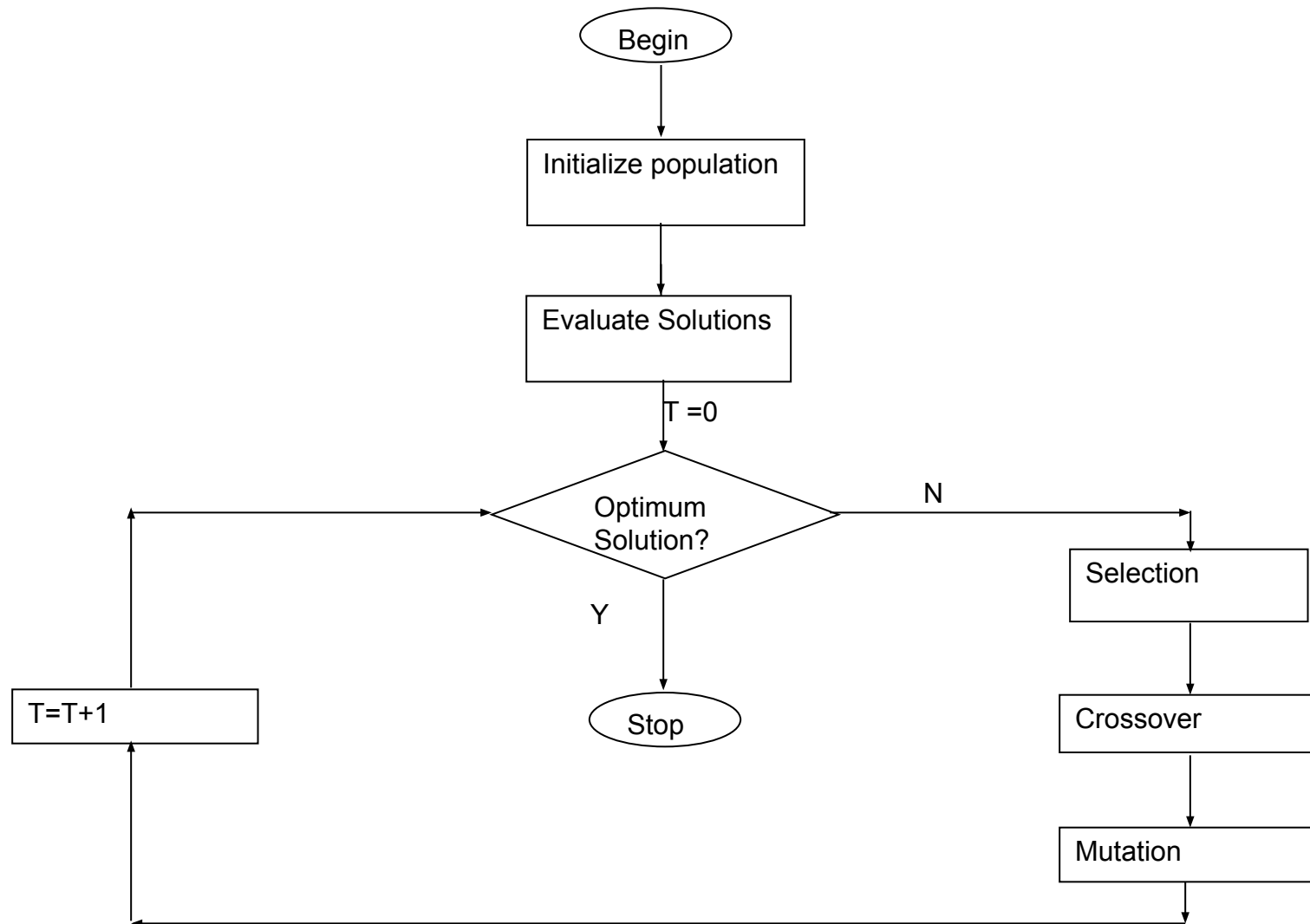
GA - Basics

- The evolution usually starts from a population of randomly generated individuals and happens in generations.
- In each generation, the fitness of every individual in the population is evaluated, multiple individuals are selected from the current population (based on their fitness), and modified (recombined and possibly mutated) to form a new population.

GA – Basics

- The new population is then used in the next iteration of the algorithm.
- Commonly, the algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.
- If the algorithm has terminated due to a maximum number of generations, a satisfactory solution may or may not have been reached.

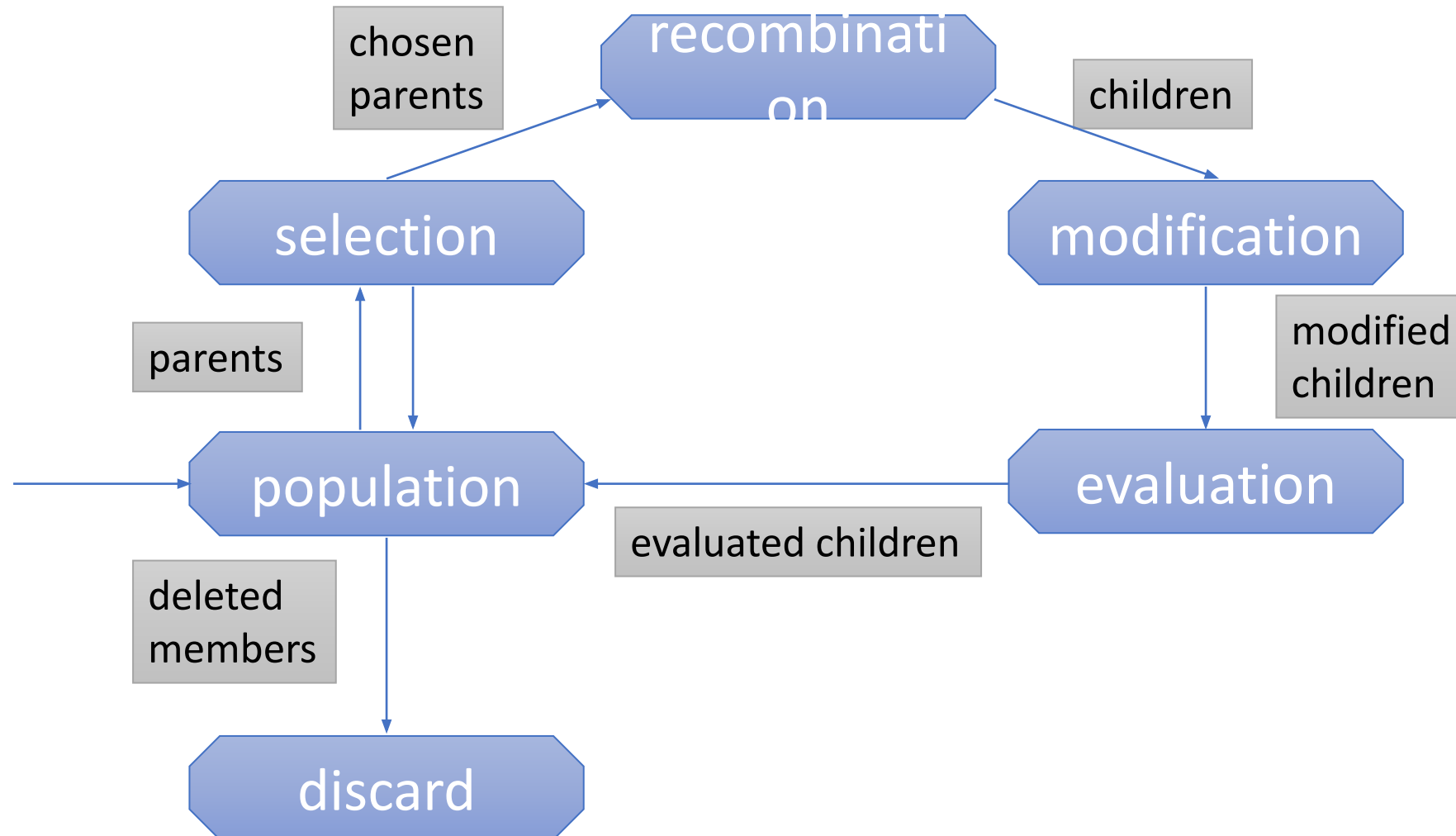
Working Mechanism Of GAs



Basic Terminologies

- **Population:** Collection of individuals (solutions)
- **Chromosome:** Encoded version of a solution
- **Gene:** Part/attribute of a chromosome
- **Allele:** Actual value of a gene
- **Fitness Function:** Objective function value for each individual
- **Generation:** One iteration of GA evolution
- **Encoding Schemes:** Binary, real-valued, permutation-based (e.g., TSP)

The GA cycle



GA Operators

- Genetic Algorithm operators mimic natural evolution. They transform a population to create new, improved generations.
- Main operators:
 - **Selection Operator**
 - **Crossover Operator**
 - **Mutation Operator**

These operators balance:

- **Exploration** (searching new areas)
- **Exploitation** (refining good solutions)

Selection Operator

Selection chooses **which individuals become parents**. Main objectives are:

- Promote **better individuals** (high fitness)
- Maintain **diversity**
- Drive evolutionary pressure toward optimal solutions

Selection pressure controls the trade-off:

- High pressure → Faster convergence but risk of premature convergence
- Low pressure → More exploration but slower improvement

Fitness Function

A fitness function quantifies the optimality of a solution (chromosome) so that that particular solution may be ranked against all the other solutions.

- A fitness value is assigned to each solution depending on how close it actually is to solving the problem.
- Ideal fitness function correlates closely to goal + quickly computable.
- Example. In TSP, $f(x)$ is sum of distances between the cities in solution. The lesser the value, the fitter the solution is.

Roulette Wheel Selection (Fitness Proportionate)

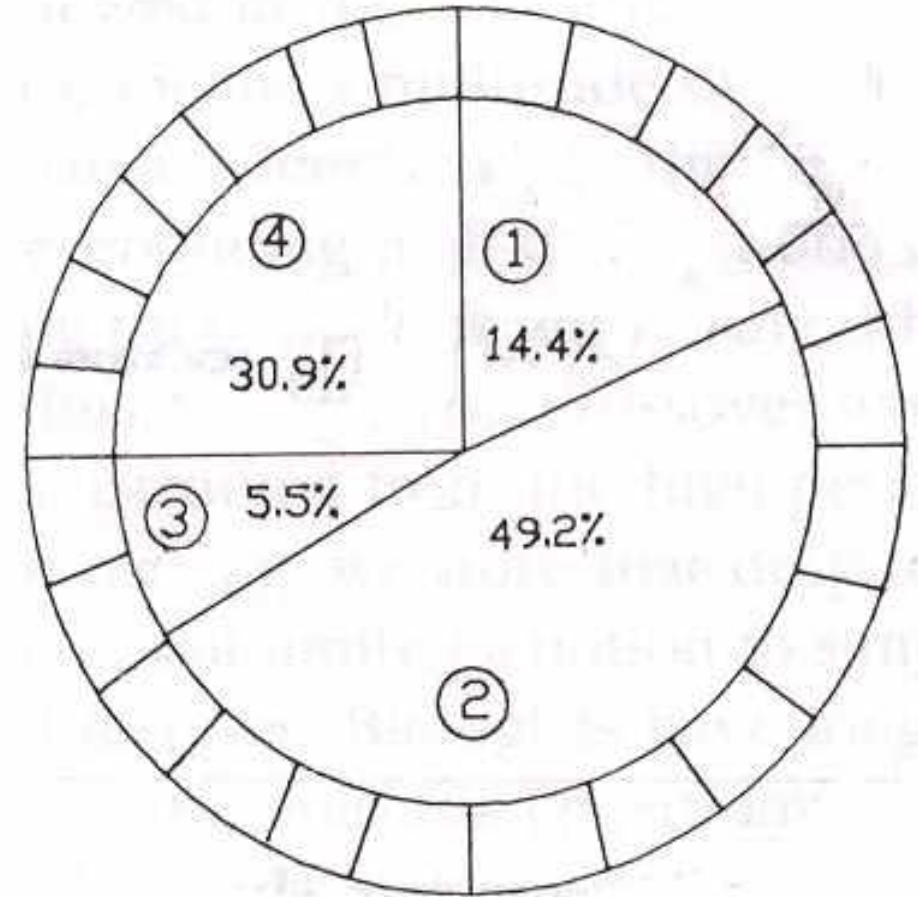
Each individual gets a slice of a roulette wheel proportional to its fitness.
The wheel is spun → random selection based on fitness probability.

Steps:

- Compute total fitness of all individuals.
- Compute probability = (individual fitness / total fitness).
- Perform cumulative probability mapping.
- Randomly generate a number in $[0, 1]$.
- Select individual whose cumulative probability interval contains the random number.

Example Of Roulette Wheel Selection

No.	String	Fitness	% Of Total
1	01101	169	14.4
2	11000	576	49.2
3	01000	64	5.5
4	10011	361	30.9
Total		1170	100.0



Roulette Wheel Selection (Fitness Proportionate)

Advantages:

- Simple and intuitive
- Good for proportional selection

Limitations:

- Sensitive to fitness scaling
- Weak pressure when fitness differences are small
- High chance of selecting mediocre individuals in noisy environments

Tournament Selection

Select **k** individuals randomly → the fittest wins. This is repeated until parent pool is formed.

Steps:

- Randomly choose k individuals (k = tournament size).
- Compare their fitness.
- Select the one with highest fitness.
- Repeat until enough parents are selected.

Tournament Selection

Advantages:

- Easy to implement
- Works with any fitness distribution
- Adjustable pressure (larger k $\rightarrow$ higher pressure)
- Better for noisy problems

Limitations:

- May lose diversity if k is too large

Rank Selection

Individuals are sorted by fitness, and selection probability is assigned by **rank**, not raw fitness.

Steps:

- Sort population by fitness.
- Assign rank 1 to worst, N to best.
- Probability = rank / sum of all ranks.
- Select based on assigned probabilities

Rank Selection

Advantages:

- Prevents domination by very high fitness values
- Maintains stable selection pressure

Limitations:

- Slower convergence compared to roulette wheel
- Does not use actual fitness differences

Elitism

Ensures the **best individuals survive unchanged** to next generation.

Purpose:

- Prevents loss of best-so-far solution
- Improves convergence speed

When used:

- Almost always included in practical GAs
- Typically top 1–5% copied to next generation

Crossover Operators

Crossover recombines genetic material from parents to create new offspring. It is the process in which two chromosomes (strings) combine their genetic material (bits) to produce a new offspring which possesses both their characteristics. The goals are:

- Exchange useful sub-structures between solutions
- Exploit building blocks
- Generate high-quality new solutions

Types depend on **encoding**:

- Binary
- Real-valued
- Permutation-based (TSP)

Crossover

- Crossover between 2 good solutions **MAY NOT ALWAYS** yield a better or as good a solution.
- Since parents are good, probability of the child being good is high.
- If offspring is not good (poor solution), it will be removed in the next iteration during “Selection”.

Encoding

- The process of representing the solution in the form of a **string** that conveys the necessary information.
- Just as in a chromosome, each gene controls a particular characteristic of the individual, similarly, each bit in the string represents a characteristic of the solution.

Encoding Methods - Binary Encoding

Most common method of encoding. Chromosomes are strings of 1's and 0's and each position in the chromosome represents a particular characteristic of the problem.

Chromosome A	10110010110011100101
Chromosome B	11111110000000011111

Encoding Methods - Value Encoding

Used in problems where complicated values, such as real numbers, are used and where binary encoding would not suffice.

Good for some problems, but *often necessary to develop some specific crossover and mutation techniques for these chromosomes.*

Chromosome A	1.235 5.323 0.454 2.321 2.454
Chromosome B	(left), (back), (left), (right), (forward)

Encoding Methods - Permutation Encoding

Useful in ordering problems such as the Traveling Salesman Problem (TSP). Example. In TSP, every chromosome is a string of numbers, each of which represents a city to be visited.

Chromosome A	1 5 3 2 6 4 7 9 8
Chromosome B	8 5 6 7 2 3 1 4 9

Single Point Crossover

A random point is chosen on the individual chromosomes (strings) and the genetic material is exchanged at this point.

Process:

- Select a random cut point.
- Swap genetic materials after that point between parents.

Chromosome1	11011 00100110110
Chromosome 2	11011 11000011110
Offspring 1	11011 11000011110
Offspring 2	11011 00100110110

Two-Point Crossover

Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

Process:

- Select two random cut points.
- Swap the segment between the cut points.

Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring 1	10101 00100 011110
Offspring 2	11011 11000 110110

Uniform Crossover

Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.

Process:

- Select two random cut points.
- Swap the segment between the cut points.

Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring	10111 00000 110110

NOTE: Uniform Crossover yields ONLY 1 offspring.

Crossover - Permutation Problems (TSP)

Permutation-based chromosomes require special crossovers to maintain valid solutions.

1. PMX (Partially Matched Crossover)

- Ensures **permutation integrity**
- Fixes duplicates by positional mapping

2. OX (Order Crossover)

- Preserves **relative order** of cities

3. CX (Cycle Crossover)

- Preserves **cycle structure**
- These are essential for **TSP, scheduling, routing**.

Mutation

Mutation introduces **small random genetic changes**. Main functions:

- Maintain diversity
- Prevent premature convergence
- Explore new areas of search space
- Repair lost genetic material

Mutation rate is usually **very small** (0.001–0.01).

Mutation

- For chromosomes using Binary Encoding, randomly selected bits are inverted.

Offspring	1101 1 00100 1 1 0110
Mutated Offspring	1101 0 00100 1 0 0110

NOTE: The number of bits to be inverted depends on the Mutation Probability.

Mutation Types for Binary Encoding

Bit Flip Mutation

Each selected gene flips its value. Simplest and most widely used

Example:

$1 \rightarrow 0$

$0 \rightarrow 1$

Advantage:

- Strong for exploring new solutions
- Very easy to implement

Mutation Types for Real Encoding

1. Gaussian Mutation

- Add a small random Gaussian value
- Good for fine-tuning optimization problems

2. Uniform Mutation

- Replace gene with random value within allowable range

Pros:

- Helps refine solutions
- Works well for continuous optimization

Mutation for Permutation Encoding

1. **Swap Mutation:** Swap positions of two genes.
2. **Insertion Mutation:** Remove a gene and insert it elsewhere.
3. **Scramble Mutation**
 - Randomly shuffle a subsegment.
 - Useful for routing & scheduling problems.

Recombination

The process that determines which solutions are to be preserved and allowed to reproduce and which ones deserve to die out.

- The primary objective of the recombination operator is to **emphasize the good solutions** and **eliminate the bad solutions** in a population, **while keeping the population size constant**.
- “Selects The Best, Discards The Rest”.
- “Recombination” is different from “Reproduction”.

Schema Theorem

- Schema theorem serves as the analysis tool for the GA process
- Explain why GAs work by showing the expectation of schema survival
- Applicable to a canonical GA
 - binary representation
 - fixed length individuals
 - fitness proportional selection
 - single point crossover
 - gene-wise mutation

Schema

- A **schema** is a set of binary strings that match the template for schema ***H***
- A template is made up of 1s, 0s, and *s where
 - * is the 'don't care' symbol that matches either 0 or 1

Schema Examples

- The schema $H = 10^*1^*$ represents the set of binary strings

10010, 10011, 10110, 10111

- The string '10' of length $l = 2$ belongs to $2^l = 2^2$ different schemas

10^* , 10^*0 , 10^*1 , 10^*10

Schema: Order $o(H)$

- The order of a schema is the number of its fixed bits, i.e. the number of bits that are not '*' in the schema H
- Example: if $H = 10^*1^*$ then $o(H) = 3$

Schema: Defining Length (H)

- The defining length is the distance between its first and the last fixed bits
- Example: if $H = *1*01$ then $(H) = 5 - 2 = 3$
- Example: if $H = 0****$ then $(H) = 1 - 1 = 0$

Schema: Count

- Suppose x is an individual that belongs to the schema H , then we say that **x is an instance of H**
($x \in H$)
- $m(H, k)$ denotes the number of instances of H in the k th generation

Schema: Fitness

- $f(x)$ denotes fitness value of x
- $f(H,k)$ denotes **average fitness of H** in the k -th generation

$$f(H,k) = \frac{\sum_{x \in H} f(x)}{m(H,k)}$$

Effect of Selection on Schema

- Assumption: fitness proportional selection
- Selection probability for the individual x

$$p_s(x) = \frac{f(x)}{\sum_{i=1} f(x_i)}$$

where the N is the total number of individuals

Net Effect of Selection

- The expected number of instances of H in the mating pool $M(H,k)$ is

$$M(H,k) = \frac{\sum_{x \in H} f(x)}{\bar{f}} = m(H,k) \frac{f(H,k)}{\bar{f}}$$

- **Schemas with fitness greater than the population average are likely to appear more in the next generation**

Effect of Crossover on Schema

- Assumption: single-point crossover
- Schema H survives crossover operation if
 - one of the parents is an instance of the schema H **AND**
 - one of the offspring is an instance of the schema H

Crossover Survival Examples

Consider $H = *10**$

$P_1 = 1\ 1\ 0\ 1\ 0 \in H$
 $P_2 = 1\ 0\ 1\ 1\ 1 \notin H$

$\Rightarrow$
 $S_1 = 1\ 1\ 0\ 1\ 1 \in H$ **Schema H**
 $S_2 = 1\ 0\ 1\ 1\ 0 \notin H$ **survived**

$P_1 = 1\ 1\ 0\ 1\ 0 \in H$
 $P_2 = 1\ 0\ 1\ 1\ 1 \notin H$

$\Rightarrow$
 $S_1 = 1\ 1\ 1\ 1\ 1 \notin H$ **Schema H**
 $S_2 = 1\ 0\ 0\ 1\ 0 \notin H$ **destroyed**

Crossover Operation

- Suppose a parent is an instance of a schema H . When the crossover is occurred within the bits of the defining length, it is destroyed unless the other parent repairs the destroyed portion
- Given a string with length l and a schema H with the defining length $\delta(H)$, the probability that the crossover occurs within the bits of the defining length is $\delta(H)/(l - 1)$

Crossover Probability Example

- Suppose $H = *1**0$

- We gave

- ▮ $l = 5$

- ▮ $\delta(H) = 5 - 2 = 3$

- Thus, the probability that the crossover occurs within the defining length is $3/4$

Crossover Operation

The upper bound of the probability that the schema H being destroyed is

$$D^c(H^c) \leq p_c \frac{\delta(H)}{l-1}$$

where p_c is the crossover probability

Net Effect of Crossover

The lower bound on the probability $S_c(H)$ that

H survives is

$$S_c(H) = 1 - D(H) \geq 1 - p \frac{\delta(H)}{l-1}$$

Schemas with low order are more likely to survive

Mutation Operation

- Assumption: mutation is applied gene by gene
- For a schema H to survive, all fixed bits must remain unchanged
- Probability of a gene not being changed is

$$(1 - p_m)$$

where p_m is the mutation probability of a gene

Net Effect of Mutation

The probability a schema H survives under mutation

$$(S_m(H) = (1 - p_m)^{o(H)})$$

Schemas with low order are more likely to survive

Schema Theorem

- Exp. # of Schema H in Next Generation >
- Exp. # in Mating Pool

$$(M(H, k) = m(H, k) \frac{f(H, k)}{\bar{f}})$$

- Prob. of Surviving Crossover

$$(S_c(H) \geq 1 - p_c \frac{\delta(H)}{l-1})$$

- Prob. of Surviving Mutation

$$(S_m(H) = (1 - p_m)^{o(H)})$$

Schema Theorem

- Mathematically

$$E[m(H, k+1)] \geq m(H, k) \frac{f(H, k)}{\bar{f}} \left(1 - p_c \frac{\delta(H)}{l-1} \right) (1 - p_m)^{o(H)}$$

- The schema theorem states that the schema with *above average fitness*, *short defining length* and *lower order* is more likely to survive

Hybrid Soft Computing

Soft Computing consists of methodologies that tolerate uncertainty and imprecision to achieve robustness, tractability, and low solution cost.

Hybrid Soft Computing = **Combination** of two or more soft computing techniques:

- **Neural Networks (NN)**
- **Fuzzy Logic (FL)**
- **Genetic Algorithms (GA)**
- **Evolutionary Strategies (ES)**
- **Probabilistic reasoning**

Hybrid Soft Computing

Purpose of hybridization:

- Exploit strengths
- Compensate for weaknesses
- Achieve better learning + better interpretability + better optimization

Hybrid Soft Computing – Limitations of NN, FL, and GA

Neural Networks

- ✓ Excellent learning capability
- ✗ Poor interpretability
- ✗ Sensitive to structure & initialization

Fuzzy Logic

- ✓ Human-understandable rules
- ✗ No automatic learning
- ✗ Difficult to tune membership functions

Genetic Algorithms

- ✓ Excellent global optimization
- ✗ Slow in fine tuning
- ✗ Need a good evaluation function

Architecture of a Hybrid System

1. Knowledge Base

Fuzzy rules

Membership functions

Neural weights

GA chromosomes

3. Inference/Decision Module

Fuzzy inference

Neural output

Evolutionary decision logic

2. Learning or Optimization Engine

NN learning (backpropagation)

GA-based optimization

Gradient or hybrid learning

4. Data Interface

Input processing

Output mapping

Hybridization can be:

Tight coupling (continuous interaction, e.g., ANFIS)

Loose coupling (one system optimizes another)

Neuro-Fuzzy System

- A system that integrates **Neural Networks** and **Fuzzy Logic**.
- NN learns from data (adaptive parameters)
- Fuzzy system provides linguistic rules & transparency

Properties:

- Human-interpretable reasoning
- Data-driven learning capability
- Nonlinear mapping ability

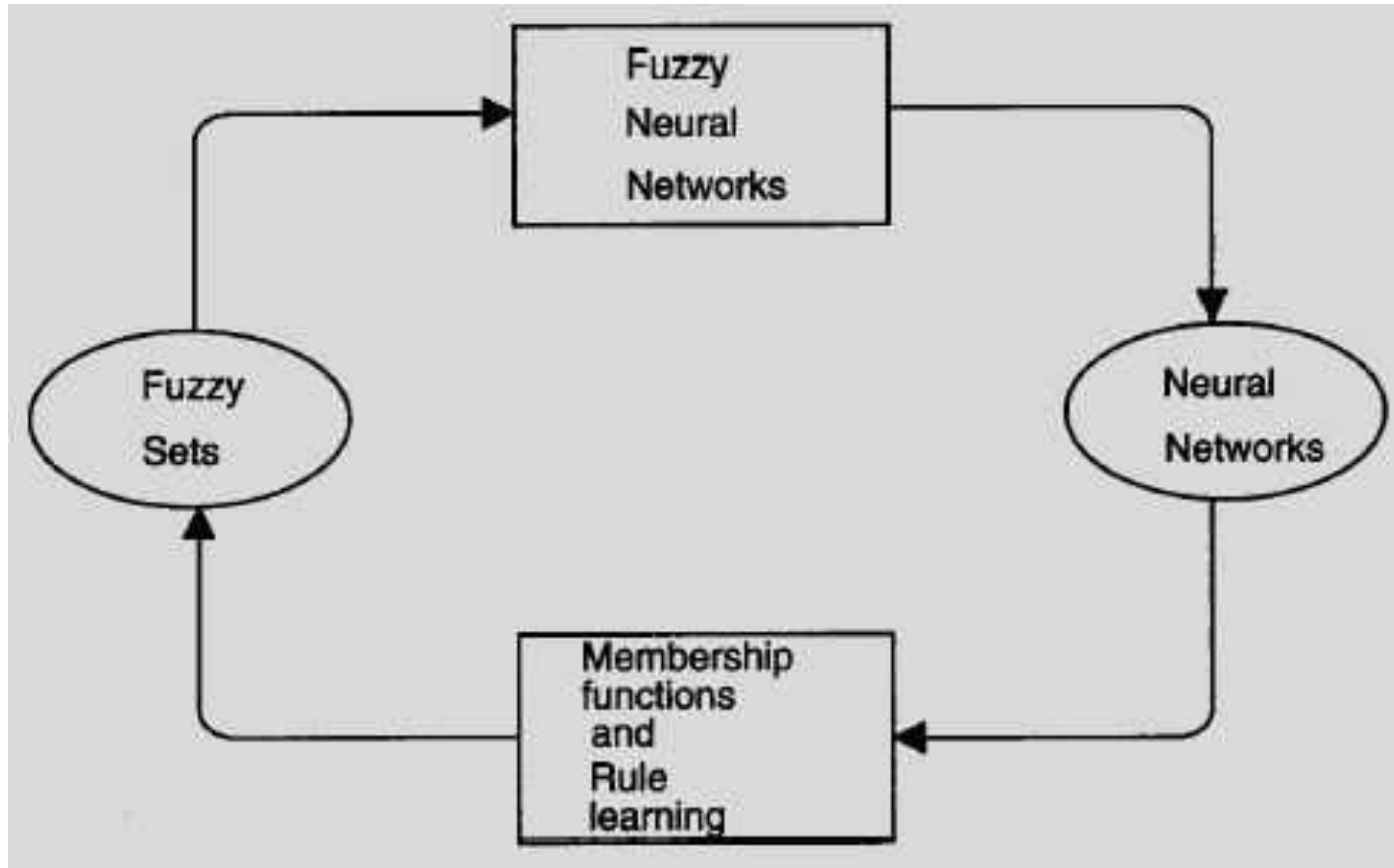
ANN – nervous system – low level perceptive and signal integration

Fuzzy – represents the emergent “higher level” reasoning aspects

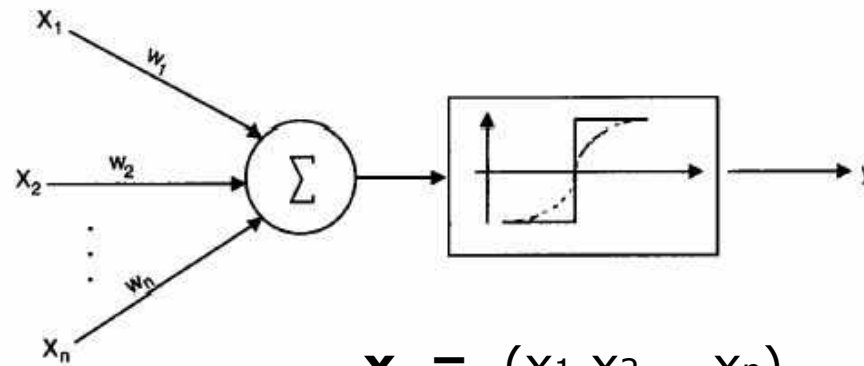
Characteristics of Neuro-Fuzzy Systems

- Use fuzzy IF–THEN rules
- Membership functions are adjusted by NN learning
- Combines:
 - ✓ Learning speed of NN
 - ✓ Interpretability of fuzzy logic
- Typical learning algorithms:
 - Backpropagation
 - Hybrid gradient + least squares (used in ANFIS)

Neuro-Fuzzy System - Architecture



Fuzzy Neurons



$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$
$$\mathbf{w} = (w_1, w_2, \dots, w_n)$$

$$y = g(\mathbf{w} \cdot \mathbf{x})$$

↓

$$y = g(A(\mathbf{w}, \mathbf{x}))$$

- Instead of weighted sum of inputs, more general aggregation function is used
- Fuzzy union, fuzzy intersection and, more generally, *s-norms* and *t-norms* can be used as an aggregation function for the weighted input to an artificial neuron

Learning in Fuzzy Neural Networks

- Supervised learning in FNN consists in modifying their connection weights in a such a manner that an error measure is progressively reduced
- Its performance should remain acceptable when it is presented with new data

Set of training data pairs $(\mathbf{x}_k, d_k)$ for $k=1,2..n$

- $\mathbf{w}^{t+1} = \mathbf{w}^t + \Delta \mathbf{w}^t$, where weight change is a given function of difference between the target response d and calculated node output y $\Delta \mathbf{w}^t = F(|d^t - y^t|)$
- **Mean square error E** – measure of how well the fuzzy network maps input data into the corresponding output $E(\mathbf{w}) = \frac{1}{2} \sum (d_k - y_k)^2$
- **Gradient descent $\Delta w_{i,j}$**
$$= -\eta \frac{\partial E}{\partial w_{ij}}$$

Neuro-Fuzzy System - Example

- **Input Layer:** passes input variables
- **Fuzzification Layer:** generates membership values
- **Rule Layer:** computes firing strength of fuzzy rules
- **Normalization & Consequent Layer:** normalizes and applies rule outputs
- **Defuzzification Layer:** produces crisp output

This layered structure resembles a neural network → trainable.

Neuro-Fuzzy Systems

Advantages

- Transparent knowledge representation
- Good generalization capability
- Automatically learn rules and MFs
- Handle nonlinearities
- Suitable for adaptive control systems
- Human-friendly interpretation

Disadvantages

- Large number of parameters to tune
- High computational cost for large rule bases
- Risk of overfitting if not regularized

Introduction to ANFIS

ANFIS is a popular model combining Takagi–Sugeno Fuzzy Inference System with the structure of a neural network.

Purpose:

- Learn fuzzy rules + membership function parameters
- Achieve high accuracy
- Preserve interpretability

Uses hybrid learning:

- Gradient descent (adjust antecedent parameters)
- Least-Squares Estimation (adjust consequent parameters)

ANFIS Architecture

Layer 1 – Fuzzification: Each node creates membership values using MFs (Gaussian, bell shaped, triangular)

Layer 2 – Rule Layer: Nodes multiply membership values → firing strength

Layer 3 – Normalization: Divides each firing strength by total strength

Layer 4 – Consequent Layer: Applies Sugeno rule:

$$\text{If } x \text{ is } A_1 \text{ and } y \text{ is } B_1 \text{ then } f = p_1x + q_1y + r_1$$

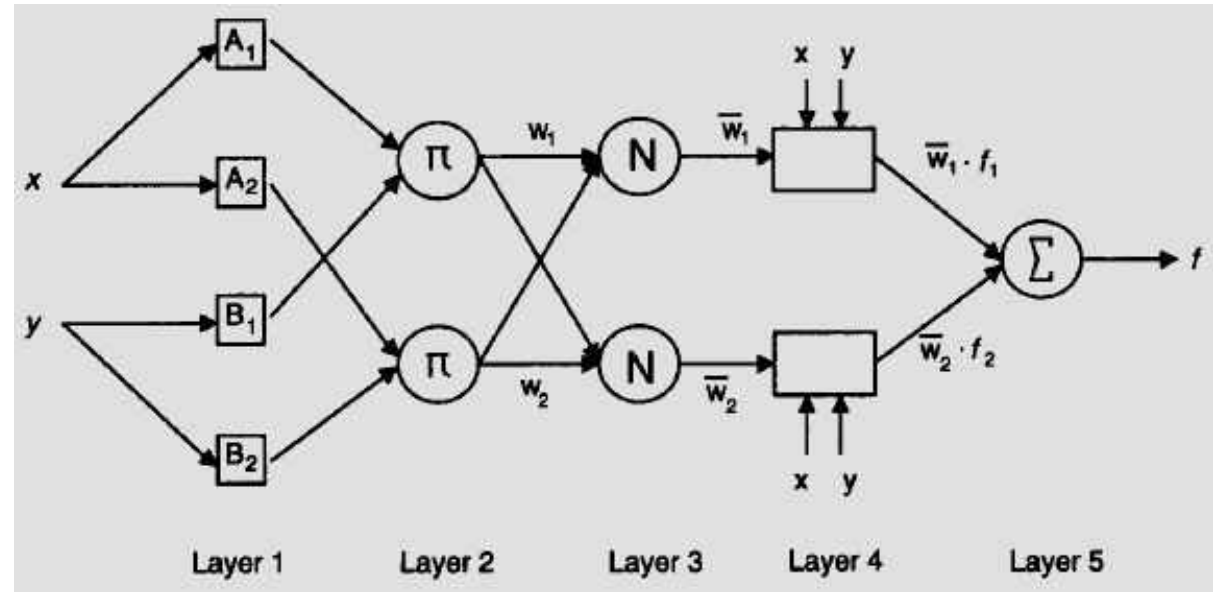
Layer 5 – Output Layer: Aggregates all rule outputs

ANFIS System - Example

IF x is A_1 AND y is B_2 THEN $f_1 = p_1x + q_1y + r_1$
IF x is A_2 AND y is B_2 THEN $f_2 = p_2x + q_2y + r_2$

The reasoning mechanism for this model is:

$$f = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2} = \bar{w}_1 + \bar{w}_2.$$



Nodes have the same function for a given layer but are different from one layer to the next

ANFIS Learning Algorithm

Two phases in each epoch:

1. Forward Pass

- Consequent parameters estimated using **Least Squares Method**
- Antecedent parameters fixed

2. Backward Pass

- Antecedent parameters tuned using **Gradient Descent**
- Consequent parameters fixed

This hybrid approach ensures:

- Fast convergence
- High accuracy

Genetic-Neuro Systems

A system where Genetic Algorithms optimize Neural Networks.

Used to improve:

- Weights
- Biases
- Network architecture
- Activation parameters

Avoids problems of backpropagation:

- Local minima
- Slow convergence
- Sensitivity to initial weights

Genetic-Neuro Systems Algorithm - Steps

1. Encode NN parameters into chromosome
2. Generate initial GA population
3. Evaluate fitness using NN performance
4. Apply selection, crossover, mutation
5. Decode chromosome → NN weights
6. Train or evaluate
7. Continue until best architecture/weights found

Genetic-Neuro Systems

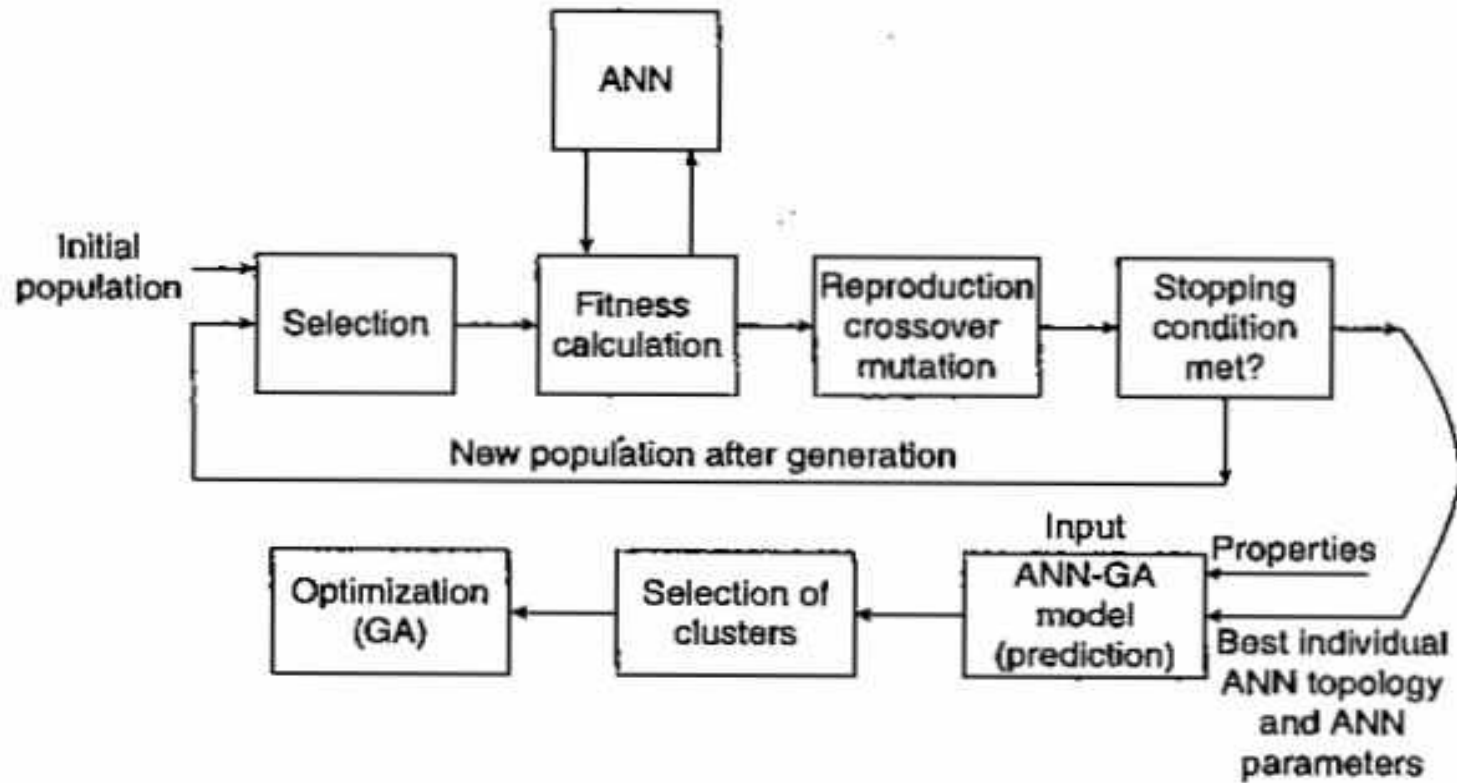


Figure 16-5 Block diagram of genetic–neuro hybrids.

Neuro-Genetic Algorithm - Advantages

- Global search → avoids local minima
- No derivative needed
- Works for non-differentiable activation functions
- Can search both structure + weights
- Supports pruning and constructive learning

Genetic-Fuzzy Systems

GA is used to optimize components of Fuzzy Logic Systems:

1. Membership Function Optimization

- Shapes, positions, widths
- Automatically tuned based on fitness

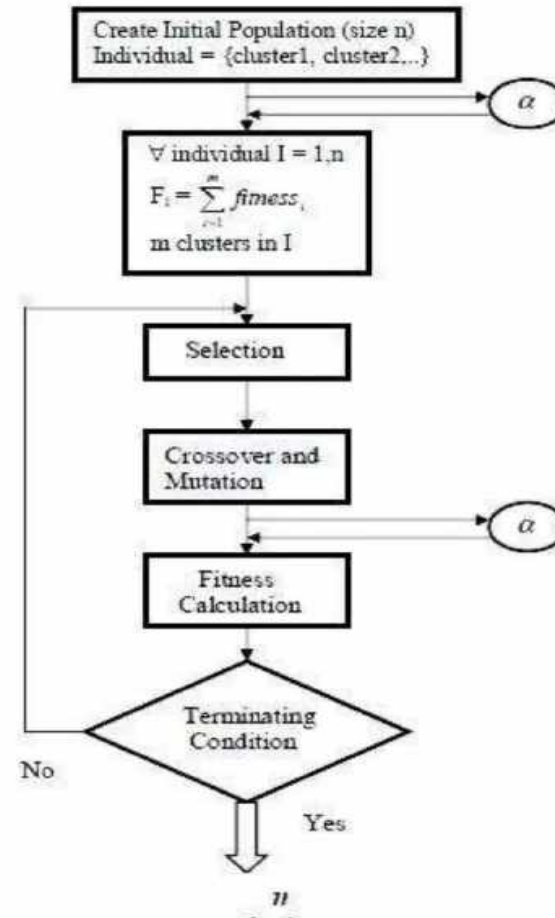
2. Rule Optimization

- Select useful rules
- Remove redundant rules
- Generate new rules

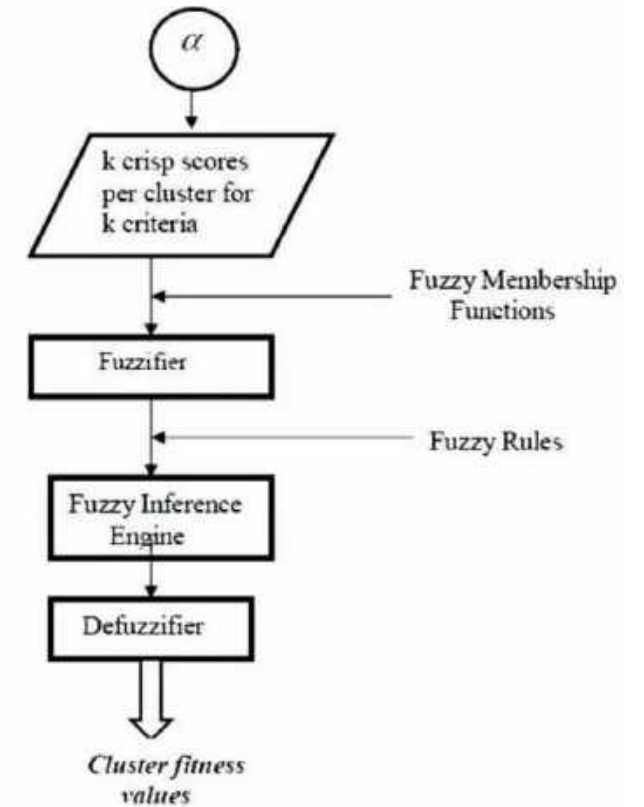
3. Structural Optimization

- Number of rules
- Variables in rules
- Granularity of fuzzy partitions

FGA - Flowchart



Flowchart of Fuzzy Guided Genetic Algorithm (GA Part)



Flowchart of Fuzzy Guided Genetic Algorithm (Fuzzy Part)

Genetic-Fuzzy Systems

Fuzzy Part

- While the population of the genetic algorithm undergoes evolution at every generation, the relatively 'good' solutions reproduce while the relatively 'bad' solutions die.
- To distinguish between solutions, an objective (evaluation) function is used. In the simple cases, there is only one criterion for optimization for example, maximization of profit or minimization of cost.
- But in many real-world decision making problems, there is a need for simultaneous optimization of multiple objectives.

Genetic Algorithm Part

- A genetic representation for potential solutions to the problem.
- Method to create an initial population of potential solutions
- An evaluation function to rate solutions in terms of their "fitness"
- Selection of individuals for the next generation
- Genetic operators that alter the composition of the children
- In order to make a successful run of a GA, the values for the parameters of the GA have to be defined like the population size, parameters for the genetic operators and the terminating condition.

Encoding Strategies for Genetic-Fuzzy

Chromosome representation can include:

- MF parameters
- Rule base
- Number of linguistic terms
- Logical operators (AND/OR)

Example:

- Gaussian MF represented as [mean, variance]
- Rule conditions represented as index positions